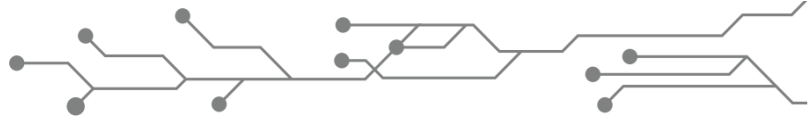


Aprendizado de Máquinas

Rodolpho Neves



1 Introdução ao aprendizado por reforço

O aprendizado por reforço (AR) é um paradigma de aprendizado no campo da inteligência artificial (IA), baseado na ideia de que agentes podem aprender comportamentos otimizados por meio de interações com o ambiente para maximizar uma recompensa cumulativa. Inspirado em como os seres vivos aprendem por tentativa e erro, o AR é amplamente utilizado em situações onde a solução ótima não é conhecida e o aprendizado é guiado por interações.

Diferentemente do aprendizado supervisionado, onde pares de entrada e saída são fornecidos, o AR utiliza recompensas como sinal de feedback para ajustar sua política de decisão. Esse modelo é amplamente utilizado em cenários onde decisões precisam ser tomadas sequencialmente, como robótica, jogos, e sistemas autônomos. As recentes inovações em AR, como o uso de redes neurais profundas em algoritmos como Deep Q-Learning, têm ampliado consideravelmente o escopo de suas aplicações.

Entre os paradigmas de aprendizado, aprendizado supervisionado, não supervisionado e aprendizado por reforço, podemos destacar algumas características:

Supervisionado: O modelo aprende com um conjunto de dados rotulados, buscando minimizar o erro entre as previsões e os valores reais.

Não supervisionado: O modelo explora dados sem rótulos para encontrar padrões ou estruturas, como agrupamentos.

Por reforço: Diferente dos outros, o AR não aprende com exemplos diretos. Ele aprende experimentando, recebendo feedback na forma de recompensas ou penalidades.

O aprendizado por reforço é uma área do aprendizado de máquina focada em como agentes podem aprender a tomar decisões por meio de interações com um ambiente. O objetivo principal é descobrir uma sequência de ações que maximize uma recompensa acumulada ao longo do tempo. O aprendizado é inspirado em como os seres humanos aprendem a partir de tentativa e erro. Por exemplo, ao aprender a andar de bicicleta, ajustes são feitos constantemente com base em recompensas (andar sem cair) ou punições (perder o equilíbrio). Esse paradigma tem aplicações em diversos campos, como:

Jogos: Em 2016, o AlphaGo, desenvolvido pelo DeepMind, venceu o campeão mundial de Go, demonstrando a capacidade do AR de lidar com problemas complexos e estratégias.

Robótica: Robôs aprendem a realizar tarefas como abrir portas ou manipular objetos frágeis utilizando o AR para refinar suas ações com base em feedback.

Carros Autônomos: O AR ajuda a treinar veículos para navegar em ambientes reais, aprendendo a evitar obstáculos e tomar decisões seguras.

Finanças: Aplicações em carteiras de investimento utilizam AR para maximizar retornos, considerando riscos associados.

A Tabela 1 apresenta uma comparação entre as características destes diferentes tipos de aprendizado, como o formato dos dados, o objetivo de cada paradigma, exemplos de aplicação e de algoritmos.

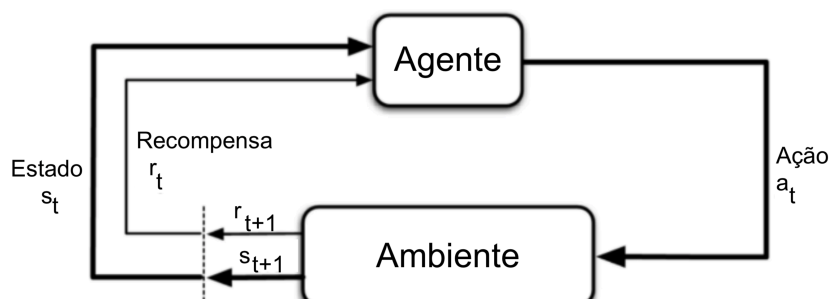
Tabela 1: Comparação entre aprendizado supervisionado, não supervisionado e por reforço.

Característica	Supervisionado	Não supervisionado	Por reforço
Dados	Dados rotulados	Dados não rotulados	Observações e recompensas
Objetivo	Predição de rótulos	Descoberta de padrões	Maximizar recompensa
Exemplo de aplicação	Classificação de e-mails (spam/não spam)	Segmentação de clientes	Jogos e controle de processos
Exemplo de algoritmo	Regressão linear, árvores de decisão	K-means, análise de componentes principais (PCA)	Q-Learning e Deep Q-Learning

1.1 Principais fundamentos do aprendizado por reforço

A Figura 1 apresenta um diagrama de um sistema de AR e seus principais elementos. É possível definir estes elementos fundamentais como:

Figura 1: Diagrama de um sistema de aprendizado por reforço.



Fonte: Adaptado de Lanham [1].

Agente: A entidade que aprende, toma decisões e realiza uma determinada ação (ex.: um robô ou um jogador).

Ambiente: O sistema com o qual o agente interage (ex.: um tabuleiro ou um espaço físico).

Estados: Representação da situação atual s_t do ambiente (ex.: posição de um robô em uma sala, posição das peças em um tabuleiro de xadrez ou as cartas em uma jogada de *blackjack*). Depois de realizada uma ação, o ambiente muda para um novo estado s_{t+1} .

Ações: Conjunto de escolhas disponíveis para o agente (ex.: mover para frente ou girar, movimentar uma determinada peça ou decidir de compra mais cartas ou pede para ver).

Recompensas: Recompensa fornecido ao agente após executar uma ação a_t a partir de um determinado estado s_t (ex.: +1 por acertar um alvo ou -1 por errar).

Política: Estratégia que mapeia o domínio dos estados s para as ações a e está presente dentro da lógica do agente e normalmente é representada por $\pi(s)$. É aqui que o agente toma suas decisões, a partir de um treinamento baseado nas recompensas ao longo do tempo.

O processo de treinamento de um agente de AR ocorre durante um episódio. Este episódio pode ser definido como uma sequência de interações do agente com o ambiente, desde um estado inicial até um estado terminal. Estas interações acontecem durante um horizonte de tempo, que pode ser finito, com um estado terminal definido, ou infinito, sem término explícito.

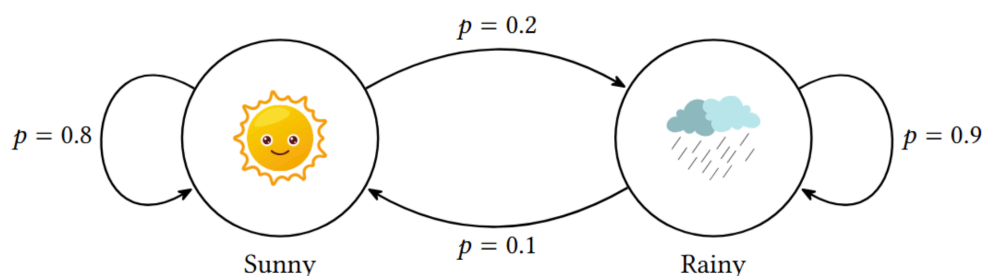
1.2 Modelo de ambiente: Processos de Decisão de Markov

Os ambientes utilizados para treinamento dos agentes no AR são, em grande maioria, simulações de processos reais. Aqui, é possível imaginar que, se um agente fará várias tentativas em alcançar o sucesso ao interagir com o ambiente e falhará na maior parte das vezes durante o processo de aprendizagem, a melhor escolha é colocar em um ambiente virtual e controlado. Ninguém quer arruinar um experimento de alguns milhões de reais porque o carro autônomo não conseguiu diminuir a velocidade em uma parada obrigatória, não é mesmo?

Nos ambientes simulados, uma das técnicas mais utilizadas para realizar a interação entre agente e ambiente é o processo de decisão de Markov (do inglês Markov Decision Process, MDP). Um processo de decisão de Markov é uma estrutura matemática que modela a interação agente-ambiente.

Um exemplo de simples de um processo de Markov [2], sem pensar em recompensas, mas com os princípios das observações e probabilidade de transição de estados é apresentado na Figura 2. Neste processo, é possível notar que existem dois estados, o ensolarado e o chuvoso. Se o ambiente se encontra

Figura 2: Modelo de tempo em forma de um processo de Markov.

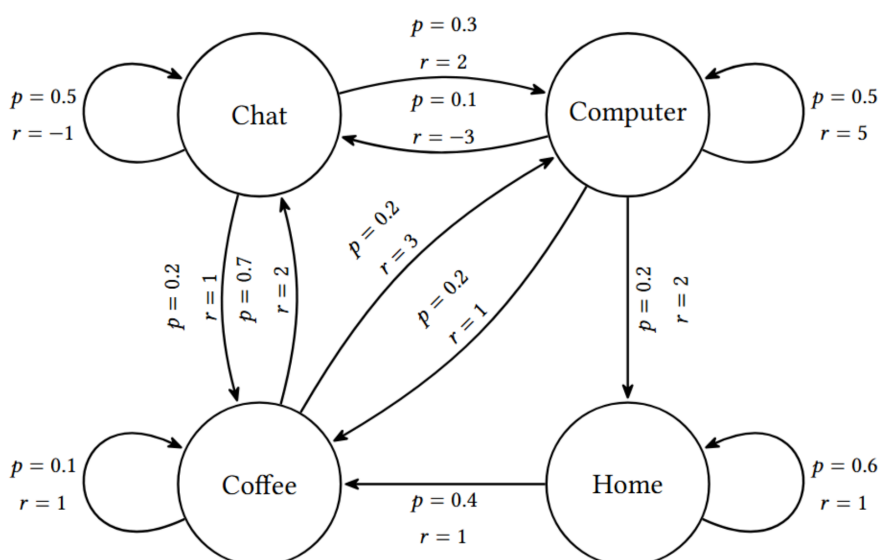


Fonte: Extraído de Lapan [2].

no processo ensolarado, a probabilidade de permanecer ensolarado é de 80% e a de mudar de estado para chuvoso é de 20%. Da mesma forma, se o processo estiver no estado chuvoso, a probabilidade de permanecer no estado chuvoso é de 90%, e de mudar, 10%. No caso deste processo, não existe um agente que pode interagir com ele, porque é um processo apenas de observação e não de decisão.

Outro exemplo de processo de Markov, mas que tenha uma entrada de decisão e, conseqüentemente, de de ação de um agente pode ser visto na Fig. 3. Este grafo representa um processo de decisão do

Figura 3: Modelo de grafo de transição de estados com probabilidades e recompensas.



Fonte: Extraído de Lapan [2].

personagem de Dilbert, o engenheiro mal-humorado que vive de forma sedentária e trabalha em uma empresa de tecnologia na Califórnia (EUA), que se resume em tomar “Café”, “Conversar” (embora ele prefira computadores à pessoas), trabalhar no “Computador” e ir/ficar em “Casa”. Cada um destes termos são os estados deste MDP. Além das probabilidades da transição para cada um dos estados, ainda está representado as recompensas de cada transição.

Outra forma de visualizar este MDP, além do grafo, é em forma de tabela, cruzando todos os estados entre si. A Tabela 2 representa o MDP, no qual estão presentes as probabilidades da transição do estado s_t para o estado s_{t+1} do Dilbert, conforme a Fig. 3.

A partir do MDP da Figura 3, é possível fazer as seguintes definições:

- Estados (S): conjuntos de todos os estados s do processo.
- Ações (A): conjunto de todas as ações a do agente.
- Modelo de transição ($P(s'|s, a)$): Probabilidade de transitar para o estado s' dado o estado atual s e ação a . Em algumas literaturas também podemos encontrar os termos $P(s_{t+1}|s_t, a_t)$ [3].
- Recompensas ($R(s, a)$): Valor esperado de recompensa ao realizar a ação a no estado s .

Tabela 2: Matriz de probabilidades de transição para o MDP do Dilbert

s_t / s_{t+1}	Home	Coffee	Chat	Computer
Home	60%	40%	0%	0%
Coffee	0%	10%	70%	20%
Chat	0%	20%	50%	30%
Computer	20%	20%	10%	50%

Fonte: Extraído de Lapan [2].

O objetivo principal do agente é maximizar o retorno de recompensa acumulado ao longo do tempo, definido como

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}, \quad (1)$$

no qual G_t é o retorno acumulado a partir do tempo t , T é o número de passos que serão realizados e γ é o fator de desconto ($0 \leq \gamma \leq 1$), que equilibra o peso de recompensas futuras em relação às imediatas. É importante destacar que nesta soma, a medida que o número de iterações agente-ambiente acontecem, mais as recompensas passadas são levadas em consideração no cálculo da recompensa acumulada G_t . Em outras palavras, quanto menor o tamanho de γ , mais as recompensas imediatas terão influência no resultado final [4].

Dado que um ambiente seja um MDP, este possui as seguintes propriedades:

Markovianidade: O próximo estado depende apenas do estado atual e da ação tomada, e não de estados anteriores. Em termos matemáticos, temos que

$$P(s_{t+1} | s_t, a_t, s_{t-1}, \dots) = P(s_{t+1} | s_t, a_t). \quad (2)$$

Estacionariedade: As regras de transição e recompensa não mudam com o tempo.

1.2.1 Função de valor

Abordagens de funções de valores de estados são baseadas no valor esperado de recompensas dado um determinado estado. As funções de valor de estado $V(s)$ mede o valor esperado de estar no estado s , seguindo uma política π , tal que

$$V_\pi(s) = \mathbb{E}[G_t | S_t = s]. \quad (3)$$

A política ótima π^* possui uma função estado-valor ótima correspondente $V^*(s)$ e vice-versa, definida por [4]

$$V^*(s) = \max_{\pi} V^\pi(s) \quad \forall s \in \mathbb{S}. \quad (4)$$

A interpretação da função de valor de estados é que, depois de um número muito grande de episódios, número de vezes que um processo é realizado, será possível, para uma política ótima π^* , escolher a ação que leva ao estado com maior recompensa. Vale ressaltar que em um MDP, o agente não conhece o valor da recompensa que será retornado para o estado. Por isso, a função de valor de estado deve ser atualizada para cada vez que o agente passe por aquele estado, de forma a, depois de inúmeras passagens, a esperança do valor de estado converge para o valor da recompensa daquele estado.

Exemplo de função de valor com o MDP do Dilbert Vamos calcular o valor esperado da função valor para cada estado do MDP da Fig. 3. Utilizando (3), vamos calcular seu valor esperado para todos os estados.

Como estamos tratando de probabilidades, vamos combinar as recompensas de cada transição com suas probabilidades. Temos que:

$$\begin{aligned} V(s = \text{"Chat"}) &= R(s = \text{"Chat"}) * P(s_{t+1} = \text{"Chat"} | s_t = \text{"Chat"}) + \\ &\quad R(s = \text{"Coffee"}) * P(s_{t+1} = \text{"Coffee"} | s_t = \text{"Chat"}) + \\ &\quad R(s = \text{"Computer"}) * P(s_{t+1} = \text{"Computer"} | s_t = \text{"Chat"}) \\ &= -1 * 0,5 + 2 * 0,3 + 1 * 0,2 = 0,3 \\ V(s = \text{"Coffee"}) &= 2 * 0,7 + 1 * 0,1 + 3 * 0,2 = 2,1 \\ V(s = \text{"Home"}) &= 1 * 0,6 + 1 * 0,4 = 1,0 \\ V(s = \text{"Computer"}) &= 5 * 0,5 + (-3) * 0,1 + 1 * 0,2 + 2 * 0,2 = 2,8 \end{aligned}$$



Estes são os valores de recompensa que o agente poderia esperar (combinados com as probabilidades) ao atingir qualquer um destes estados, considerando um $\gamma = 0$. Se $\gamma = 1$ e o processo rodar infinitamente, a função valor tende a infinito, porque este MDP não possui um estado de finalização, como uma linha de chegada. Valores de γ entre 0 e 1 levaria a função valor de cada estado para um valor constante, dependendo de quantos passos fossem executados neste MDP.

1.3 Política de ação

Se até agora o agente apenas observou as transições de estado em um MDP, a partir daqui ele terá que escolher alguma ação para manipular o processo de transição de estados no ambiente. Como você já deve imaginar, a partir da Tabela 2, vimos que a transição de um estado gera uma tabela de probabilidades em que eles ocorrem. A adição das ações do agente a esta tabela resulta em uma tabela com três dimensões (duas pelos estados e uma pelas ações).

As políticas de ação do agente é que definem como ele vai se comportar a partir de um determinado estado. Este é o principal elemento do aprendizado por reforço e tem como objetivo maximizar a recompensa do agente. Portanto, diferentes políticas de ações podem resultar em diferentes acumulados de recompensas. Matematicamente, a política de ação é definida por

$$\pi(a|s) = P(A_t = a|S_t = s), \quad (5)$$

na qual A_t é o conjunto de ações a que o agente pode realizar e S_t , o conjunto de s estados em que ele está. Como pode ser notado, a política é uma probabilidade de realizar uma ação.

Existem duas abordagens principais para a construção de algoritmos de aprendizado por reforço, abordagens baseadas em função de valores de estados (*value functions*) esperados de recompensas e abordagens baseadas em buscas políticas (*policy searches*). Existem ainda abordagens híbridas de função de valores e busca política, como a *actor-critic*, mas não serão cobertas neste curso. Vamos entender como funcionam o Q-Learning e o Deep Q-Learning, algoritmos baseados em *value functions* e em *policy searches*.

2 Q-Learning: Aprendizado por tabela

O Q-Learning é uma técnica popular de aprendizado por reforço que permite a um agente aprender a escolher as melhores ações para maximizar sua recompensa acumulada ao longo do tempo. É uma abordagem para problemas discretos finitos, no qual podemos contar os estados do ambiente e o número de ações que o agente pode realizar [2, 3].

Podemos citar algumas vantagens do Q-Learning, como a capacidade de aprender diretamente da interação com o ambiente. Outra vantagem é não exigir conhecimento prévio do modelo de transição do ambiente ($P(s'|s, a)$).

Por ser uma abordagem baseada em função de valores de estados, o Q-learning relaciona os estados às ações do agente, construindo uma tabela para encontrar as melhores ações. A principal ideia é usar uma tabela $Q(s, a)$ para armazenar valores que estimam o quão bom é executar a ação a em um estado s . Com o tempo, essa tabela é preenchida e reflete a política ótima, ou seja, as melhores ações para cada estado [1, 2].

2.1 Atualização da tabela Q

O Q-Learning é baseado em uma fórmula de atualização iterativa. Quando o agente realiza uma ação a em um estado s e observa a recompensa r e o próximo estado s' , a tabela $Q(s, a)$ é atualizada por

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right], \quad (6)$$

na qual $Q(s, a)$ é o valor atual armazenado na tabela para o par estado-ação, α é a taxa de aprendizado ($0 < \alpha \leq 1$), r é a recompensa imediata observada, γ é o fator de desconto ($0 \leq \gamma \leq 1$), que controla a importância de recompensas futuras e $\max_{a'} Q(s', a')$ é a estimativa do valor ótimo do próximo estado s' .

O termo dentro dos colchetes representa o erro temporal (*temporal difference error*, ou TD-error), que mede a diferença entre a estimativa atual e a recompensa observada mais o valor futuro esperado.

O pseudocódigo do Q-learning pode ser definido como:

1. Inicializar $Q(s, a)$ arbitrariamente para todos os estados s e ações a ;
2. Para cada episódio:
 - (a) Inicializar o estado inicial s ;
 - (b) Enquanto s não for um estado terminal:
 - i. Escolher ação a baseada em uma estratégia
 - ii. Executar a ação a e observar a recompensa r e o próximo estado s'
 - iii. Atualizar $Q(s, a)$ usando:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

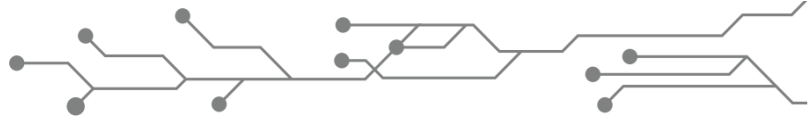
- iv. $s \leftarrow s'$

É importante definir dois termos que aparecem no pseudocódigo, o episódio e o estado terminal. Os episódios são as vezes que o algoritmo pode buscar por melhores soluções, de forma iterativa. Ele inicia com o estado inicial do problema e finaliza a partir do estado terminal. O estado terminal pode ser o agente alcançar o objetivo final do problema, como em um caso de labirinto, encontrar a saída. Outra forma de estado terminal é alcançar o número máximo de episódios permitidos.

A escolha da estratégia pelo agente, caso siga sempre a melhor combinação que ele conheça, pode estar limitada pelo seu conhecimento sobre o ambiente. Uma estratégia interessante de ser alternada com a tabela Q é uma ação aleatória. O agente decide se vai usar a tabela Q ou a ação aleatória no Passo 2.b.i do pseudocódigo, a partir de uma chance ϵ de ser aleatória. Esta estratégia é denominada ϵ -greedy.

2.1.1 Dilema da exploração x exploração

Se o agente mantém estimativas sobre os valores das ações, então, em qualquer passo, pelo menos umas das ações com valores estimados é a que possui maior valor. Esta estratégia é denominada política



da ação gananciosa, do inglês *greedy action*. Se a política de *greedy action* está ativa, é dito que o agente está explotando o conhecimento atual dos valores das ações, pensando sempre na ação que traz a maior recompensa naquele momento. Entretanto, se a política de ação for uma política não gananciosa (do inglês *nongreedy actions*), é dito que o agente está explorando.

Na exploração, o agente tem a possibilidade de descobrir, e melhorar, suas estimativas quanto às possíveis ações que ele poderia realizar. Com isso, em geral, a política *nongreedy actions* alcança recompensas acumuladas melhores do que as políticas *greedy action*, por permitir a exploração ao invés da exploração [3]. Esta estratégia deve ser dosada para que o agente não assuma uma estratégia totalmente aleatória, como na maioria das políticas *nongreedy actions*.

2.2 Limitações do Q-Learning

Embora o Q-Learning seja poderoso, ele apresenta algumas limitações:

Escalabilidade: A tabela $Q(s, a)$ cresce exponencialmente com o aumento do número de estados e ações, tornando-se inviável para ambientes complexos.

Espaços de estado contínuos: Não é diretamente aplicável a problemas onde estados e/ou ações são contínuos, como em robótica. Nesses casos, métodos como Deep Q-Learning são mais adequados.

Exploração limitada: Estratégias simples como ϵ -greedy podem não explorar suficientemente bem ambientes complexos.

Referências

- [1] M. Lanham, *Hands-on reinforcement learning for games: implementing self-learning agents in games using artificial intelligence techniques*. Packt Publishing Ltd, 2020.
- [2] M. Lapan, *Deep reinforcement learning hands-on*. Packt Publishing Ltd, 2024.
- [3] R. S. Sutton, “Reinforcement learning: An introduction,” *A Bradford Book*, 2018.
- [4] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, “Deep reinforcement learning: A brief survey,” *IEEE Signal Processing Magazine*, vol. 34, no. 6, pp. 26–38, 2017.